

Parami 기획 보완 문서

날씨 기반 자동 보상 구독 서비스

1. 현황 파악 및 Pain Point 도출

1.1 시장 현황

영역	현황
기존 날씨 보험	B2B 위주 (농업·이벤트·항공). 개인 대상 상품은 사실상 없음.
일반 구독 서비스	정량적 혜택이 모호해 가치 체감이 어려움.
적금·예금	보상은 명확하나 일상과의 연결고리가 없음.
날씨 데이터 활용 서비스	정보 제공(예보·미세먼지 알림)에 머무름. 보상으로 연결되지 않음.

1.2 사용자 Pain Point

- **악천후가 발생해도 개인이 받을 수 있는 즉각적 보상은 존재하지 않음.** 폭우·폭염·한파·미세먼지·폭설은 일상의 비용(세탁비, 교통비, 마스크, 난방비)을 발생시키지만 보전 장치가 없음.
- **사후 청구 방식의 마찰.** 기존 보험은 피해 입증 → 증빙 수집 → 청구 → 심사 → 며칠~수 주 후 보상의 흐름이 일반적. 개인이 일상적인 날씨 피해를 청구하기엔 비용이 더 큼.
- **긍정 인센티브의 부재.** 모든 시스템이 "악조건 발생 → 보상"에만 초점. 좋은 날씨라는 일상의 가치를 보상으로 환원하는 시도는 없음.
- **사용자 능동성 요구.** 사용자가 날씨를 직접 확인하고 행동해야 가치를 얻을 수 있는 구조.

1.3 운영자 측 Pain Point

- 보험 청구 심사를 위한 인력·비용 부담.
- 케이스 바이 케이스 판단으로 인한 정책 일관성 저하.
- 사용자 활동 데이터 부재 — 어떤 사용자가 어떤 조건에서 가장 큰 가치를 느끼는지 측정 어려움.

2. AS-IS vs TO-BE 모델 및 가치 제안

2.1 AS-IS — 기존 방식

[사용자]
└ 날씨 앱 직접 확인
└ 불편 / 피해 감수
└ (보험 가입 시) 증빙 수집 → 청구서 작성 → 제출
 └ [운영자] 심사 → 사후 보상 (며칠~수 주)

- 사용자 능동성 100% 필요
- 보상 시점이 피해 시점과 분리됨

- 좋은 날씨에 대한 보상 경로 없음

2.2 TO-BE — Parami 구조

```
[Cloud Scheduler] 매시간 KST
└─ POST /api/scheduler/weather-check
    └─ 기상청·에어코리아 API 호출
    └─ weather_logs INSERT (시간당 누적)
    └─ 악조건 5종 + 긍정조건 판별 (lib/conditions.ts)
        └─ trigger_logs UNIQUE(trigger_type, triggered_date)
            └─ 활성 구독 + 결제 success 유저 조회
                └─ 월 10회 캡 체크
                    └─ TRANSACTION
                        reward_logs INSERT
                        users.balance += amount
                        COMMIT

[사용자]
└─ 구독만 유지 → 대시보드에서 자동 적립된 보상 확인
```

- 사용자 능동성 0 (구독 유지만)
- 보상 시점 = 조건 충족 시점 (실시간)
- 긍정 조건(좋은 날씨)도 보상 가능

2.3 가치 제안

사용자 가치

- 청구 절차 없는 자동 보상 — 마찰 제거.
- 일상의 불편이 즉시 가치로 환원되는 체감.
- 좋은 날씨 보상으로 일상 자체에 대한 긍정 강화 (봄·가을 한정).
- 월 10회 캡으로 예측 가능한 혜택 구조.

운영자 가치

- 청구 심사 인력·비용 0 — 100% 자동화.
- 정책 일관성 — 모든 트리거가 동일 로직으로 판정.
- 데이터 누적 — `weather_logs`, `trigger_logs`, `reward_logs`로 행동 데이터 확보.
- 외부 데이터(기상청/에어코리아) 기반이라 운영자가 임의로 조작 불가 → 신뢰 확보.

3. 차별점 및 전략

3.1 기존 서비스와의 차별점

비교축	기존 보험·구독	Parami
보상 방식	사후 청구	자동 트리거
청구 마찰	증빙·심사 필요	없음

비교축	기존 보험·구독	Parami
보상 시점	며칠~수 주 후	조건 충족 즉시
보상 조건	악조건 한정	악조건 + 긍정조건
데이터 출처	사용자 신고	공공 API (기상청·에어코리아)
사용자 능동성	필수	불필요
정책 일관성	케이스별 판단	코드 기반 결정론적 판정

3.2 전략

기술 전략

- **Next.js (App Router) 풀스택** — 프론트와 API를 한 코드베이스에서 관리해 인원 부담 자유롭게.
- **MySQL + UNIQUE 제약** — 중복 트리거·중복 보상을 DB 레벨에서 차단. 앱 로직 실수에도 안전.
- **GCP Cloud Scheduler** — 매시간 실행, 외부 의존성 단일화. 자체 cron보다 안정.
- **토스페이먼트 단건 결제** — 정기결제 인프라 없이도 구독 모델 운영 가능. 결제 신뢰성 확보.
- **하드코딩된 트리거 조건** ([lib/conditions.ts](#)) — 정책 변경 시 코드 리뷰를 거치게 해 일관성 유지.

운영 전략

- 월 10회 캡으로 보상 비용 상한 확정.
- [weather_logs](#) 시간당 누적 → 사후 감사 가능.
- 모든 트리거에 UNIQUE 제약 → 동일 조건 중복 발동 차단.
- 활성 구독 + 최근 결제 success 동시 충족자만 보상 → 결제 분쟁 시 추적 가능.

4. 요구사항 정의 및 파이프라인 전략

4.1 요구사항 정리

기능 요구사항

영역	내용
회원	가입, 로그인, 로그아웃, 내 정보 조회
결제	토스 단건 결제 승인, 결제 내역 조회
구독	티어 변경, 해지, 상태 조회
날씨	시간당 자동 수집 (기상청 + 에어코리아)
트리거	6종 조건 판별 (rain/heat/cold/snow/dust + good_weather)
보상	자동 지급, 월 10회 캡, 잔액 적립
관리자	유저·구독·결제·트리거·보상 전 항목 조회

데이터 요구사항

- 외부: 기상청 단기예보 API, 에어코리아 PM2.5·PM10 API

- 내부: 7개 테이블 (users, user_accounts, subscriptions, payments, weather_logs, trigger_logs, reward_logs)
- 누적: **weather_logs**는 시간당 누적, 절대 삭제 금지 (사후 감사용)

인증·권한 요구사항

- 자체 JWT 발급 (소셜 로그인도 추후 확장)
- HttpOnly 쿠키로 토큰 저장 (XSS 방어)
- **role = 'admin'** 별도 체크로 관리자 API 보호
- 스케줄러 엔드포인트는 **SCHEDULER_SECRET** 헤더로 외부 호출 차단

운영 요구사항

- Cloud Scheduler 매시간 호출 보장
- 외부 API 장애 시 graceful degradation
- DB 트랜잭션으로 보상 정합성 보장
- 모든 환경변수는 **.env.local**에 두고 절대 커밋 금지

4.2 파이프라인 전략

전체 데이터 흐름

1. Cloud Scheduler 발화 (매시간 KST)
2. POST /api/scheduler/weather-check (헤더 인증)
3. 기상청 API 호출 → 강수량·기온·풍속·PTY
4. 에어코리아 API 호출 → PM2.5·PM10
5. weather_logs INSERT (raw_payload 포함)
6. 악조건 판별 (rain≥3 / heat≥33 / cold≤-12 / dust≥50 / snow PTY 3·4)
 - ↳ 충족 → trigger_logs UNIQUE 시도
 - ↳ 중복(이미 오늘 발동) → 종료
 - ↳ 신규 → trigger_logs 저장 → 7단계로
7. 긍정조건 판별 (현재 월 ∈ [4,5,6,9,10,11] AND rain≤1 AND pm25≤30 AND wind≤5)
 - ↳ 동일하게 UNIQUE 시도 후 분기
8. 트리거 신규 발동 시:
 - ↳ 활성 구독 + 최근 결제 success 유저 조회
 - ↳ 월 캡 체크 (reward_year + reward_month + user_id 기준 KST)
 - ↳ BEGIN TRANSACTION
 - reward_logs INSERT
 - users.balance = balance + amount
 - COMMIT
 - ↳ 실패 시 ROLLBACK
9. 응답 반환 (성공/실패 카운트 로그)

예상 병목 지점과 대응

단계	병목	대응
3·4 외부 API	응답 지연·장애	타임아웃 5초 + 1회 재시도. 실패 시 해당 시간 스킵하고 다음 시간에 재시도.

단계	병목	대응
5 INSERT	트래픽 자체는 미미	인덱스 <code>idx_measured_at</code> , <code>idx_location_measured_at</code> 으로 조회 가속
6-7 UNIQUE 충돌	중복 호출로 인한 충돌	UNIQUE 제약으로 DB 레벨에서 처리. 충돌 시 정상 분기.
8 유저 조회	사용자 증가 시 N+1.전체 스캔	활성 구독 인덱스, 페이지네이션-배치 처리 검토
8 보상 트랜잭션	balance 동시 갱신	TRANSACTION + <code>users</code> 행 단위 락. ROLLBACK 시 <code>reward_logs</code> 도 함께 무효화.
MySQL NULL UNIQUE	<code>toss_payment_key</code> NULL 다 수 허용됨	앱 레벨 SELECT 중복 체크로 보강 (CLAUDE.md 명시)

5. 사용자 흐름 설계

5.1 전체 사용자 여정

[비회원]

- ↳ 메인 페이지 / 상품 소개
 - ↳ 회원가입 (POST /api/auth/signup)
 - ↳ `users` INSERT + `user_accounts(provider='local')` INSERT
 - ↳ JWT 발급 → HttpOnly 쿠키 저장
- ↳ 로그인 (POST /api/auth/login)
 - ↳ 비밀번호 검증 → JWT 발급

[로그인 상태]

- ↳ 티어 목록 조회 (GET /api/plans)
- ↳ 결제 (토스 SDK 호출 → POST /api/payments/confirm)
 - ↳ 결제 승인 + `payments` INSERT + `subscriptions` INSERT (동일 트랜잭션)
- ↳ 대시보드 진입
 - ↳ GET /api/subscriptions/me
 - ↳ GET /api/rewards/me
 - ↳ GET /api/rewards/summary
- ↳ (백그라운드 자동) Cloud Scheduler가 매시간 트리거 체크
 - ↳ 조건 충족 시 `reward_logs` INSERT + balance 증가
- ↳ 다음 진입 시 누적 보상 확인

[옵션]

- ↳ 티어 변경 (PATCH /api/subscriptions/change-tier)
- ↳ 구독 해지 (PATCH /api/subscriptions/cancel)
- ↳ 로그아웃 (POST /api/auth/logout) → 쿠키 삭제

[관리자]

- ↳ `role='admin'` 유저만 /api/admin/* 접근 가능
- ↳ 유저·구독·결제·트리거·보상 전수 조회

5.2 인증·인가 적용 기술 및 처리 구조

인증 (Authentication) — "누구인가"

- 자체 JWT (HS256). 비밀키는 `JWT_SECRET` 환경변수.
- 회원가입·로그인 성공 시 7일 만료 토큰 발급.
- 토큰은 HttpOnly + Secure 쿠키에 저장 → 클라이언트 JS에서 접근 불가 → XSS 방어.
- 매 요청마다 `lib/auth.ts`의 `getSession(request)`가 쿠키에서 토큰을 추출하고 `verifyToken()`으로 검증.

인가 (Authorization) — "무엇을 할 수 있는가"

- 일반 API Route: `getSession()` 결과로 `user_id`를 얻어 본인 데이터만 조회·수정.
- 관리자 API Route (`/api/admin/*`): `getSession()` + `users.role === 'admin'` 추가 검증.
- 스케줄러 API Route (`/api/scheduler/*`): JWT 대신 `SCHEDULER_SECRET` 헤더로 검증. Cloud Scheduler가 호출 시 헤더 자동 부착.

처리 구조 (요청 → 응답)

요청 도착

- ↳ Next.js Middleware 또는 API Route에서 `getSession(request)` 호출
 - ↳ 쿠키 'token' 추출 → JWT verify
 - ↳ 실패 → 401 { success: false, error: '인증 필요' }
 - ↳ 성공 → payload.user_id 확보
- ↳ (관리자 API라면) DB에서 role 확인
 - ↳ role !== 'admin' → 403
- ↳ 비즈니스 로직 수행 (`lib/db.ts` 풀에서 쿼리)
- ↳ 응답 { success: true, data: ... }

6. 정리

- **Pain Point:** 개인 대상 즉시 보상 부재 + 청구 마찰 + 긍정조건 보상 부재.
- **TO-BE 핵심:** 외부 데이터 → 스케줄러 → 자동 트리거 → 자동 보상.
- **차별점:** 자동·즉시·긍정조건 포함·결정론적 정책.
- **요구사항·파이프라인:** 7개 테이블 + UNIQUE 제약 + TRANSACTION으로 정합성 보장.
- **사용자 흐름:** 구독 유지만으로 가치 발생. 인증은 JWT + HttpOnly 쿠키, 스케줄러는 별도 시크릿 헤더.